In [1]:
```
import torch
import torch.nn as nn
import snntorch as snn
import snntorch.functional as SF
import snntorch.surrogate as surrogate

import numpy as np
import pandas as pd
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import train_test_split
from sklearn.metrics import classification_report, confusion_matrix,
roc_auc_score
```

In [2]:
```
# =====================
#  🔹 Dataset preparation
# =====================
df = pd.read_csv("..//../../creditcard.csv")

# Fraudes e não fraudes
fraud = df[df["Class"] == 1]
non_fraud = df[df["Class"] == 0]
```

In [3]:
```
# Conjunto de treino (390 fraudes + 5000 não-fraudes)
fraud_train = fraud.sample(n=390, random_state=42)
nonfraud_train = non_fraud.sample(n=5000, random_state=42)
train_data = pd.concat([fraud_train, nonfraud_train])

# Conjunto de teste (101 fraudes + 999 não-fraudes)
fraud_test = fraud.drop(fraud_train.index).sample(n=101, random_state=42)
nonfraud_test = non_fraud.drop(nonfraud_train.index).sample(n=999,
random_state=42)
test_data = pd.concat([fraud_test, nonfraud_test])

# Separar features e labels
X_train = train_data.drop("Class", axis=1).values
y_train = train_data["Class"].values
X_test = test_data.drop("Class", axis=1).values
y_test = test_data["Class"].values
```

In [4]:
```
# Normalização (StandardScaler)
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
```

In [5]:
```
# Dividir treino/validação (20%)
X_train, X_val, y_train, y_val = train_test_split(
    X_train, y_train, test_size=0.2, random_state=42, stratify=y_train
)
```

In [6]:
```
# Convertendo para tensores
X_train = torch.tensor(X_train, dtype=torch.float32)
y_train = torch.tensor(y_train, dtype=torch.long)
X_val = torch.tensor(X_val, dtype=torch.float32)
y_val = torch.tensor(y_val, dtype=torch.long)
X_test = torch.tensor(X_test, dtype=torch.float32)
y_test = torch.tensor(y_test, dtype=torch.long)
```

```
# DataLoaders
batch_size = 64
train_loader = torch.utils.data.DataLoader(
    torch.utils.data.TensorDataset(X_train, y_train),
    batch_size=batch_size, shuffle=True
)
val_loader = torch.utils.data.DataLoader(
    torch.utils.data.TensorDataset(X_val, y_val),
    batch_size=batch_size, shuffle=False
)
test_loader = torch.utils.data.DataLoader(
    torch.utils.data.TensorDataset(X_test, y_test),
    batch_size=batch_size, shuffle=False
)
```

In [14]:

```
# =====================
#  🔹 Modelo SNN (Baseline do artigo)
# =====================
num_steps = 10

class SNN_Model(nn.Module):
    def __init__(self):
        super().__init__()
        self.fc1 = nn.Linear(30, 100)  # 30 → 100
        self.lif1 = snn.Leaky(beta=0.9, spike_grad=surrogate.atan(),
init_hidden=True)
        self.lif2 = snn.Leaky(beta=0.9, spike_grad=surrogate.atan(),
init_hidden=True)
        self.fc2 = nn.Linear(100, 2)   # 100 → 2

    def forward(self, x):
        mem1 = self.lif1.init_leaky()
        mem2 = self.lif2.init_leaky()

        spk2_rec = []

        for step in range(num_steps):
            cur1 = self.fc1(x)
            #spk1, mem1 = self.lif1(cur1, mem1)
            spk1 = self.lif1(cur1)   # sem mem

            #spk2, mem2 = self.lif2(spk1, mem2)
            spk2 = self.lif2(spk1)   # idem
            out = self.fc2(spk2)
            spk2_rec.append(out)

        return torch.stack(spk2_rec).mean(0)  # Média sobre os steps
```

In [15]:

```
# =====================
#  🔹 Treinamento
# =====================
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
model = SNN_Model().to(device)
optimizer = torch.optim.Adam(model.parameters(), lr=1e-3)
criterion = nn.CrossEntropyLoss()
```

In [16]:

```
epochs = 30

for epoch in range(epochs):
```

```
    model.train()
    total_loss = 0
    for data, targets in train_loader:
        data, targets = data.to(device), targets.to(device)

        optimizer.zero_grad()
        outputs = model(data)
        loss = criterion(outputs, targets)
        loss.backward()
        optimizer.step()
        total_loss += loss.item()

    # Validação
    model.eval()
    val_loss, correct = 0, 0
    with torch.no_grad():
        for data, targets in val_loader:
            data, targets = data.to(device), targets.to(device)
            outputs = model(data)
            val_loss += criterion(outputs, targets).item()
            pred = outputs.argmax(1)
            correct += (pred == targets).sum().item()

    if (epoch + 1) % 5 == 0 or epoch == 0:
        print(f"Epoch {epoch+1}/{epochs}, Loss={total_loss/
len(train_loader):.4f}, ValAcc={correct/len(val_loader.dataset):.4f}")
```

```
Epoch 1/30, Loss=0.4084, ValAcc=0.9759
Epoch 5/30, Loss=0.0430, ValAcc=0.9805
Epoch 10/30, Loss=0.0307, ValAcc=0.9842
Epoch 15/30, Loss=0.0244, ValAcc=0.9842
Epoch 20/30, Loss=0.0197, ValAcc=0.9842
Epoch 25/30, Loss=0.0158, ValAcc=0.9842
Epoch 30/30, Loss=0.0125, ValAcc=0.9842
```

In [17]:

```
# =====================
#  🔹 Avaliação final no teste
# =====================
model.eval()
all_preds, all_labels = [], []
with torch.no_grad():
    for data, targets in test_loader:
        data, targets = data.to(device), targets.to(device)
        outputs = model(data)
        preds = outputs.argmax(1).cpu().numpy()
        all_preds.extend(preds)
        all_labels.extend(targets.cpu().numpy())

print("\n📊 Resultados no conjunto de teste:")
print(classification_report(all_labels, all_preds, digits=4))
print("Matriz de confusão:\n", confusion_matrix(all_labels, all_preds))
print("AUC:", roc_auc_score(all_labels, all_preds))
```

```
📊 Resultados no conjunto de teste:
              precision    recall  f1-score   support

           0     0.9871    0.9980    0.9925       999
           1     0.9778    0.8713    0.9215       101

    accuracy                         0.9864      1100
   macro avg     0.9825    0.9346    0.9570      1100
weighted avg     0.9863    0.9864    0.9860      1100

Matriz de confusão:
 [[997   2]
 [ 13  88]]
AUC: 0.9346425633554346
```

In [ ]: